GrBoardPagination.class

```java
// 페이징 수 계산 클래스(총 42개의 글이 있을 때 5개의 페이징이 나와야 하므로 계산하는 클래스)
@Getter  3개 사용 위치
public class GrBoardPagination {

    private int totalPage;      // 전체 페이지 몇 개?
    private int totalCount;     // 글 전체 몇 개?

    private int page;           // 현재 페이지는?
    private int recordSize;     // 현재 페이지에 표시할 글 개수는?

    private int pageSize = 10;   // 한 화면에 보여줄 페이지 버튼 개수
    private int startPage;      // 시작 페이지 번호
    private int endPage;        // 마지막 페이지 번호

    private boolean existPrevPage;  // 이전 페이지 묶음 존재 여부
    private boolean existNextPage;  // 다음 페이지 묶음 존재 여부
    private int prevPage;           // 이전 버튼을 눌렀을 때 이동할 페이지
    private int nextPage;           // 다음 버튼을 눌렀을 때 이동할 페이지

    // 페이징 수 계산
    public GrBoardPagination(GrBoardSerchRequest params, int totalCount) {  1개 사용 위치
        this.page = params.getPage(); // 요청받은 현재 페이지 (1, 2, 3...)
        this.recordSize = params.getRecordSize(); // 한 페이지에 표시할 일반글 개수 (SQL의 LIMMIT)
        this.totalCount = totalCount; // DB에서 조회한 전체 일반글 개수

        this.totalPage = (int) Math.ceil((double) totalCount / recordSize); // 총 글 개수 대비해서 몇 페이지 나오는지?

        // 페이징 수에 따른 페이징 버튼 계산
        this.startPage = ((page-1) / pageSize) * pageSize + 1;
        this.endPage = Math.min(startPage + pageSize - 1, totalPage);

        this.existPrevPage = startPage > 1; // 1보다 큰 페이지의 경우에만 이전 버튼 생성
        this.existNextPage = endPage < totalPage; // 전체 페이지보다 끝 페이지가 작을 경우에만 다음 버튼 생성
        this.prevPage = startPage - 1;
        this.nextPage = endPage + 1;
    }
```

전달받은 page와 recordSize로 페이징 버튼에 필요한 계산을 한 뒤 model에 담는다.

전달받은 page와 recordSize로 페이징 버튼에 필요한 계산을 한 뒤 model에 담는다.

html

```html
<ul class="pagination m-0" th:if="${pagination != null and pagination.totalPage > 0}">

    <!-- 이전 페이지 묶음 -->
    <li class="page-item" th:if="${pagination.existPrevPage}">
        <a class="page-link" th:href="@{/openGrBoardMain(category=${params.category}, page=${pagination.prevPage},
                                        searchType=${params.searchType}, keyword=${params.keyword})}">이전</a>
    </li>

    <!-- 페이지 번호 -->
    <li class="page-item" th:each="pageNumber : ${#numbers.sequence(pagination.startPage, pagination.endPage)}"
                          th:classappend="${pageNumber == pagination.page} ? 'active' : ''">
        <a class="page-link" th:href="@{/openGrBoardMain(category=${params.category}, page=${pageNumber}, searchType=${params.searchType}, keyword=${params.keyword})}"
                          th:text="${pageNumber}">1</a>
    </li>

    <!-- 다음 페이지 묶음 -->
    <li class="page-item" th:if="${pagination.existNextPage}">
        <a class="page-link" th:href="@{/openGrBoardMain(category=${params.category}, page=${pagination.nextPage},
                                        searchType=${params.searchType}, keyword=${params.keyword})}">다음</a>
    </li>
```

- th:each는 반복문이며 pagination.startPage부터 pagination.endPage까지 숫자 나열.
- th:classappend는 삼항연사자이며 현재 반복된 숫자가 page값과 같으면 active 클래스를 사용해 UI 변경

Controller.class

```java
// 게시판 메인페이지 호출
// params: 페이징 처리를 위한 객체 정보
@GetMapping("/openGrBoardMain")
public String openGrBoardMain(Model model, GrBoardSerchRequest params) {


    List<GrBoardResponse> grBoardList = grBoardService.grBoardList(params);

    // 페이징 버튼을 위한 전체 글 개수 조회
    int totalCount = grBoardService.grBoardCount(params);
    // 페이징 버튼을 위한 페이징 수 계산
    GrBoardPagination pagination = new GrBoardPagination(params, totalCount);


    model.addAttribute( attributeName: "grBoardList", grBoardList);
    model.addAttribute( attributeName: "pagination", pagination);
    model.addAttribute( attributeName: "params", params);

    return "grBoard/grBoardMain";
}
```

- html에서 서버를 통해 보낸 page값을 페이징 버튼 처리를 위해 그대로 재사용한다.

페이징 버튼 처리를 위한 글 전체 개수 조회 (totalCount)

service, mapper interface 생략

mapper.xml

```xml
<!-- 일반글 전체 개수  -->
<select id="grBoardCount" parameterType="io.github.yhj118.groupware.util.GrBoardSerchRequest" resultType="int">
    SELECT COUNT(*)
        FROM (
            SELECT gr.*, e.department_id, e.employee_name
                FROM gr_board gr
                INNER JOIN employee e
                    ON gr.employee_id = e.employee_id
                    WHERE gr.delete_yn = 0
            ) q1
            INNER JOIN department d ON q1.department_id = d.department_id
    <where>
        <!-- 카테고리 검색 -->
        <if test="category != null and category != '' and category != 'ALL'">
            AND q1.category = #{category}
        </if>

        <!-- 검색어가 있을 때만 검색 -->
        <if test="keyword != null and keyword != ''">
            <choose>
                <!-- 제목 + 내용 -->
                <when test="searchType == 'titleContent'">
                    AND (q1.title LIKE CONCAT('%', #{keyword}, '%') OR q1.content LIKE CONCAT('%', #{keyword}, '%'))
```